

Informática Teórica

Aula 6

Definição de Máquina de Turing (MT)

Definição

A Máquina de Turing é um modelo matemático de computação que define uma máquina abstrata capaz de manipular símbolos em uma fita de acordo com uma tabela de regras.

Embora simples, ela é capaz de simular a lógica de qualquer algoritmo de computador, servindo como a base para o que entendemos hoje por computabilidade.

Por que ela é "Abstrata"?

Dizemos que ela é abstrata porque ela é um **modelo matemático**, não um objeto físico.

- Sem Limites Físicos
- Essência da Operação
- Universalidade

Por que ela pode simular a lógica de qualquer algoritmo?

Qualquer **algoritmo** pode ser quebrado em **três operações básicas** que a MT realiza:

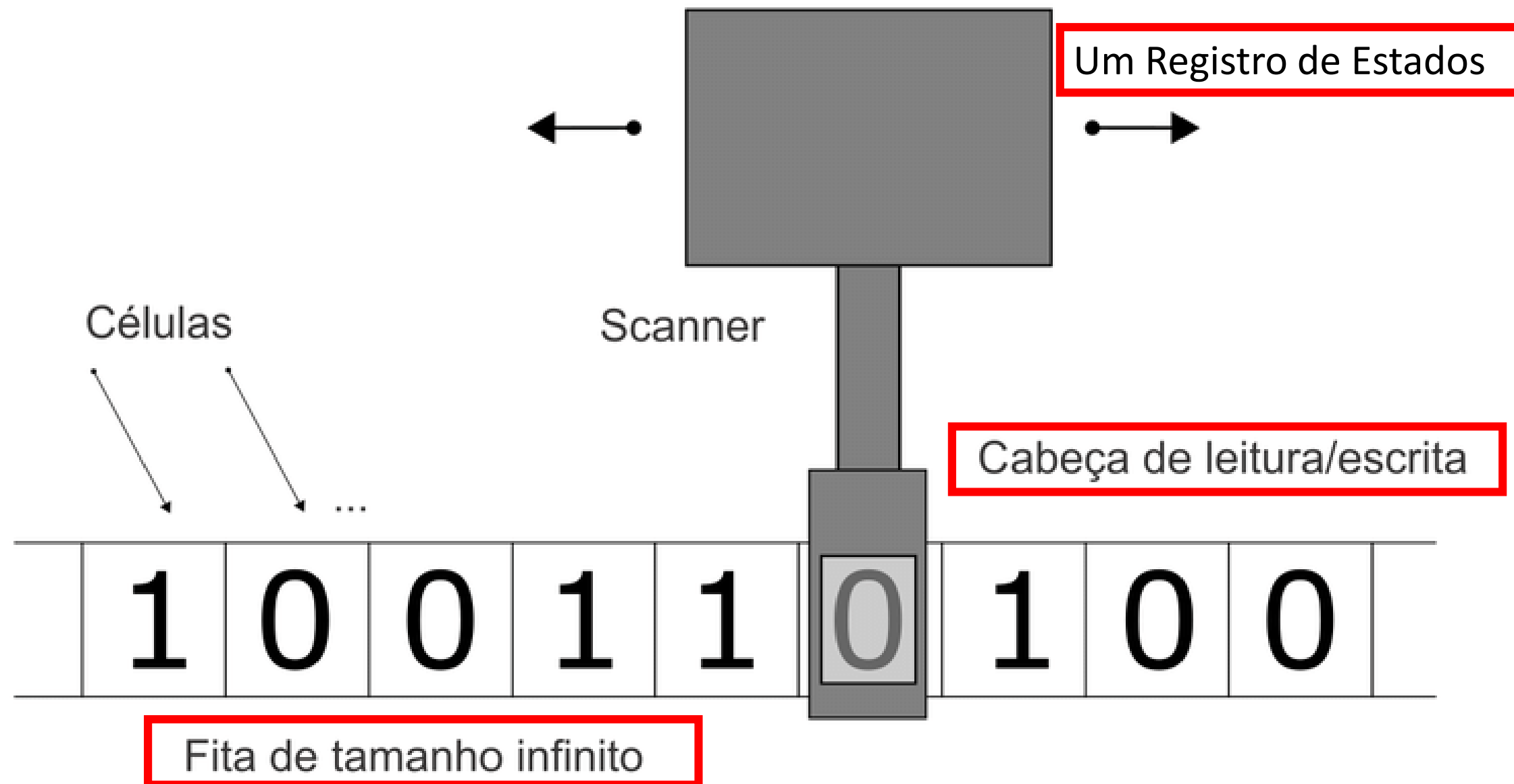
- 1.Ler/Escrever:** Mudar o estado de uma unidade de informação.
- 2.Mover:** Acessar a próxima unidade de informação.
- 3.Decidir:** Mudar o comportamento com base no que foi lido (o "if/else" primordial).

Se algo pode ser calculado por um **supercomputador da NASA**, pode ser calculado por uma **Máquina de Turing** (dado tempo e fita suficientes).

O que é Computabilidade?

A **computabilidade** é o estudo do que **pode** e do que **não pode** ser resolvido por um algoritmo.

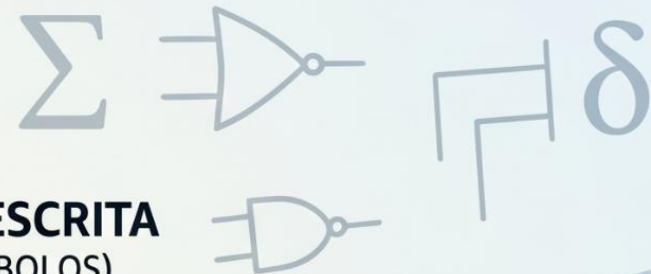
- **Problemas Computáveis:** São aqueles que a máquina consegue terminar e dar uma resposta (ex: somar dois números, ordenar uma lista).
- **Problemas Não-Computáveis:** Existem perguntas lógicas que uma máquina simplesmente entraria em um loop infinito e nunca conseguiria responder.



Algorithm notation

Algorithm:
state $\leftarrow m = \{1\}$
if $(n > 0)$
else $= 0$
stax $\leftarrow \{1\}$

MÁGUNA DE TURING



REGISTRO DE ESTADOS
(ARMAZENA O ESTADO LÓGICO)

CABEÇA DE LEITURA/ESCRITA
(MOVE-SE E MANIPULA SÍMBOLOS)

ESTADO ATUAL:
Q1 (SOMA)

DIREITA

ESQUERDA

FITA INFINITA

(DIVIDIDA EM CÉLULAS DE MEMÓRIA)

CÉLULA

(ENDEREÇO DE MEMÓRIA)

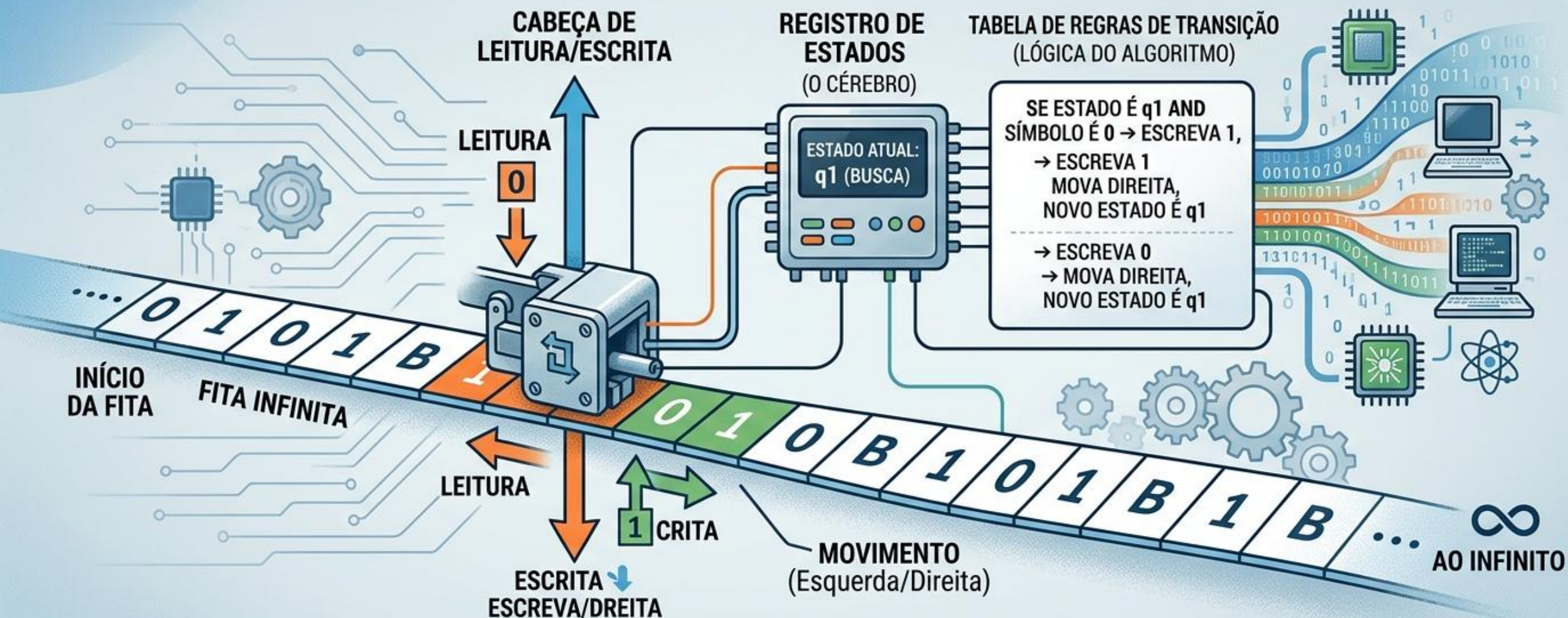
SÍMBOLO LIDO: 0

SÍMBOLOS NA FITA
(Ex: 0, 1, B)

ALGORITHMS



MÁQUINA DE TURING: O MODELO ABSTRATO DE COMPUTAÇÃO



ABSTRATA: Fita Infinita, Lógica Pura
(Sem Limites Físicos)

SIMULAÇÃO: Executa Passos Básicos
de Qualquer Algoritmo

COMPUTABILIDADE: O Que Pode e Não
Pode Ser Resolvido (Problema da Parada)

A Fita Infinita (Memória Principal)

A fita é o local onde os dados residem.

As Células: Cada quadrado na fita pode conter um único símbolo (geralmente 0, 1 ou um símbolo de "branco").

O Conceito de Infinito: Na prática, nenhuma fita é infinita, mas para a matemática, essa "infinitude" é crucial.

Entrada e Saída: A fita começa com os dados do problema (entrada) e, ao final do processo, o que sobrar escrito nela é o resultado (saída).

A Cabeça de Leitura/Escrita (O "Ponteiro")

Se a fita é o papel, a **cabeça é a caneta**. Ela é a única **interface entre o "cérebro" da máquina e a memória**.

- **Ações Limitadas:** Ela só consegue fazer três coisas por vez:
 - **Ler** o que está na célula atual
 - **Apagar e escrever** um novo símbolo
 - **Mover uma casa** para a esquerda ou para a direita.
- **Acesso Sequencial:** Para ler o milésimo símbolo, você precisa passar pelos 999 anteriores.

O Registro de Estados (O "Estado Mental")

Este é o componente mais abstrato e importante. Ele define o que a máquina está "pensando" ou qual passo do algoritmo ela está executando.

O Contexto: A máquina não tem memória de longo prazo sobre o que fez dez passos atrás. Ela só sabe em que **Estado** está agora.

Ex:

- "Estado de Busca"(**buscar símbolo**)
- "Estado de Soma"(**altera**)
- "Estado Final"(**resposta**)

Exercício de Fixação: O Inversor de Bits

1. Enunciado

Projete uma **Máquina de Turing (MT)** que receba em sua fita uma cadeia binária composta por símbolos do alfabeto $\{0, 1\}$. A máquina deve percorrer a fita, inverter cada bit (transformando 0 em 1 e 1 em 0) e encerrar a execução (aceitar) assim que encontrar o final da cadeia (símbolo branco $\sqcup$).

2. A Sétupla Formal

A máquina M é definida matematicamente pelos seguintes componentes:

- $Q = \{q_0, q_{aceita}\}$: Conjunto finito de estados.
- $\Sigma = \{0, 1\}$: Alfabeto de entrada.
- $\Gamma = \{0, 1, \sqcup\}$: Alfabeto da fita (inclui a entrada e o símbolo branco).
- q_0 : Estado inicial.
- q_{aceita} : Estado de aceitação.
- $q_{rejeita} = \emptyset$: Neste problema, não há condições explícitas de erro.
- δ : Função de transição (detalhada abaixo).

3. Definições Formais das Transições (δ)

A função δ define o comportamento exato da Unidade de Controle para cada par de (Estado, Símbolo). A regra segue o formato:

$$\delta(q_{atual}, leitura) = (q_{proximo}, escrita, direção)$$

1. Processamento do bit 0:

$$\delta(q_0, 0) = (q_0, 1, R)$$

(Se ler 0, escreve 1, permanece em q_0 e move para a direita).

$$\delta(q_{atual}, leitura) = (q_{proximo}, escrita, direção)$$

2. Processamento do bit 1:

$$\delta(q_0, 1) = (q_0, 0, R)$$

(Se ler 1, escreve 0, permanece em q_0 e move para a direita).

3. Condição de Parada (Fim da Cadeia):

$$\delta(q_0, \sqcup) = (q_{aceita}, \sqcup, L)$$

(Ao encontrar o vazio, mantém o vazio, transiciona para o estado final e move para a esquerda).

Passo a Passo
(Rastreamento da Fita)

[0][1][1][L]

$$\delta(q_0, 0) = (q_0, 1, R)$$

q_0 ↓

[0][1][1][␣]

q_0 ↓

[1][1][1][␣]

$$\delta(q_0, 1) = (q_0, 0, R)$$

q_0 ↓
[1][1][1][␣]

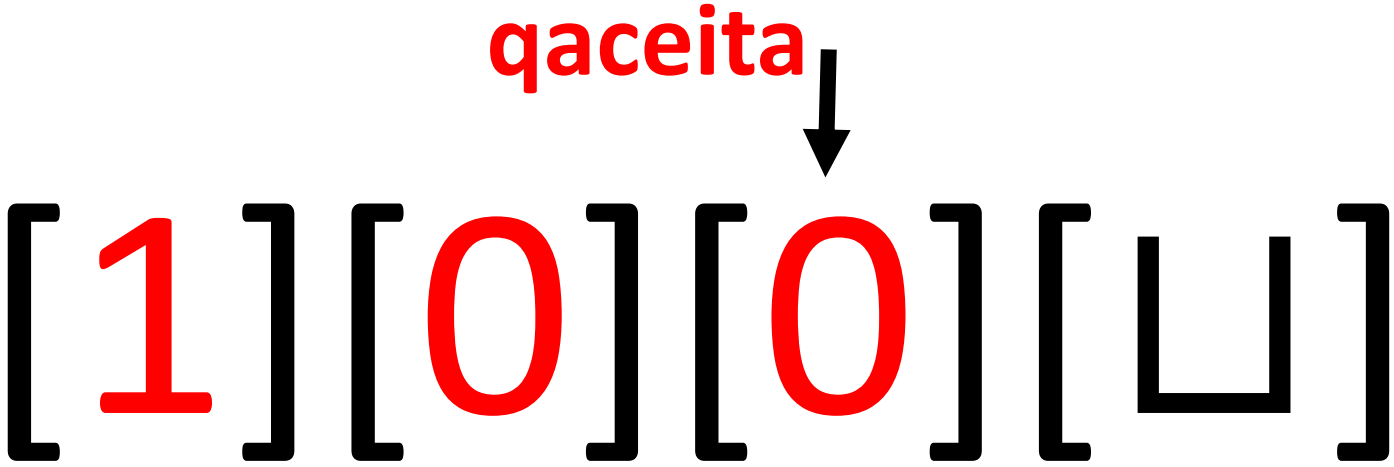
q_0 ↓
[1][0][1][␣]

$$\delta(q_0, 1) = (q_0, 0, R)$$

$q_0 \downarrow$
[1][0][1][␣]

$q_0 \downarrow$
[1][0][0][␣]

$\delta(q_0, \sqcup) = (q_{aceita}, \sqcup, L)$



Passo	Estado	Conteúdo da Fita
1	q_0	0 1 1 □
2	q_0	1 1 1 □
3	q_0	1 0 1 □
4	q_0	1 0 0 □
Fim	q_{aceita}	1 0 0 □

A Diferença Fundamental: AFD/AFN vs. MT

Diferença Fundamental: AFD/AFN vs. MT

A grande "virada de chave" para os alunos é entender que os Autômatos Finitos (AFD e AFN) são extremamente limitados em termos de memória e movimento.

Recurso	Autômatos Finitos (AFD/AFN)	Máquina de Turing (MT)
Memória	Finita e Imutável: Reside apenas nos estados.	Infinita e Editável: Possui uma fita de leitura e escrita.
Movimento	Unidirecional: Apenas para a direita (consome a entrada).	Bidirecional: Pode ir e voltar na fita livremente (L e R).
Ação	Apenas Leitura: Não pode alterar a entrada.	Leitura e Escrita: Pode modificar o conteúdo da fita.

O Que é a Tese de Church-Turing?

A Proposição Central

"Qualquer função que possa ser calculada por um algoritmo pode ser calculada por uma Máquina de Turing."

Em outras palavras, se existe um passo a passo lógico para resolver um problema, existe uma Máquina de Turing capaz de executá-lo.

Exemplos

Exemplo: Reconhecedor da linguagem $L = \{aw \mid w \in \{a, b\}^*\}$

$$\delta(q_{atual}, \text{leitura}) = (q_{proximo}, \text{escrita}, \text{direção})$$

$$\delta(q_0, a) = (q_1, a, R) \quad \delta(q_1, b) = (q_1, b, R)$$

$$\delta(q_1, a) = (q_1, a, R) \quad \delta(q_1, \sqcup) = (q_2, \sqcup, R)$$

Fita: aab

Exemplo: Reconhecedor da linguagem $L = \{aw \mid w \in \{a, b\}^*\}$

$$\delta(q_{atual}, \text{leitura}) = (q_{proximo}, \text{escrita}, \text{direção})$$

$$\delta(q_0, a) = (q_1, a, R) \quad \delta(q_1, b) = (q_1, b, R)$$

$$\delta(q_1, a) = (q_1, a, R) \quad \delta(q_1, \sqcup) = (q_2, \sqcup, R)$$

Fita: aaabbb

Exemplo: Reconhecedor da linguagem $L = \{aw \mid w \in \{a, b\}^*\}$

$$\delta(q_{atual}, \text{leitura}) = (q_{proximo}, \text{escrita}, \text{direção})$$

$$\delta(q_0, a) = (q_1, a, R) \quad \delta(q_1, b) = (q_1, b, R)$$

$$\delta(q_1, a) = (q_1, a, R) \quad \delta(q_1, \sqcup) = (q_2, \sqcup, R)$$

Diagrama de Estados

Exercício: Inversor de Bits (Transdutor)

Enunciado:

Projete uma Máquina de Turing transdutora que receba na fita uma cadeia binária qualquer $w \in \{0, 1\}^*$ e inverta todos os seus bits (trocando 0 por 1 e 1 por 0). A máquina deve processar a fita da esquerda para a direita e parar no estado de aceitação assim que encontrar o fim da palavra (símbolo de vazio $\sqcup$).

- **Alfabetos:** $\Sigma = \{0, 1\}$ e $\Gamma = \{0, 1, \sqcup\}$
- **Estados:** $Q = \{q_0, q_1\}$, onde q_0 é o inicial e q_1 é o de aceitação.

Exercício: Inversor de Bits (Transdutor)

$$\delta(q_0, 0) = (q_0, 1, R)$$

$$\delta(q_0, 1) = (q_0, 0, R)$$

$$\delta(q_0, \sqcup) = (q_1, \sqcup, R)$$

Fita: 1100

Exercício: Inversor de Bits (Transdutor)

$$\delta(q_0, 0) = (q_0, 1, R)$$

$$\delta(q_0, 1) = (q_0, 0, R)$$

$$\delta(q_0, \sqcup) = (q_1, \sqcup, R)$$

Fita: 101010

Exercício: Inversor de Bits (Transdutor)

$$\delta(q_0, 0) = (q_0, 1, R)$$

$$\delta(q_0, 1) = (q_0, 0, R)$$

$$\delta(q_0, \sqcup) = (q_1, \sqcup, R)$$

Diagrama
de
Estados

Lista de Exercícios: Praticando Máquinas de Turing

→ Rastreamento (Trace) da fita

→ Criar diagramas de estados.

3. Definições Formais das Transições (δ)

A função δ define o comportamento exato da Unidade de Controle para cada par de (Estado, Símbolo). A regra segue o formato:

$$\delta(q_{atual}, leitura) = (q_{proximo}, escrita, direção)$$

Exercício 1: O Caçador de "1"s

Enunciado: Projete uma MT que percorra a fita para localizar o primeiro símbolo 1. Ao encontrá-lo, deve substituí-lo por um X e parar em q_{aceita} . Se encontrar um branco ($\sqcup$) antes do 1, deve parar em $q_{rejeita}$.

• **Fita de Teste:** $[0][0][1][0][\sqcup]$

$$\delta(q_0, 0) = (q_0, 0, R)$$

$$\delta(q_0, 1) = (q_{aceita}, X, R)$$

$$\delta(q_0, \sqcup) = (q_{rejeita}, \sqcup, R)$$

- Rastreamento
- Diagrama de Estados

Exercício 1: O Caçador de "1"s

Enunciado: Projete uma MT que percorra a fita para localizar o primeiro símbolo 1. Ao encontrá-lo, deve substituí-lo por um X e parar em q_{aceita} . Se encontrar um branco ($\sqcup$) antes do 1, deve parar em $q_{rejeita}$.

- **Fita de Teste:** $[0][0][1][0][\sqcup]$
- **Resultado Esperado:** A fita deve terminar como $[0][0][X][0][\sqcup]$ com a máquina em q_{aceita} .

Exercício 2: Limpando a Memória

Enunciado: Crie uma MT que apague toda a entrada da fita. Ela deve substituir cada 0 ou 1 pelo símbolo branco ($\sqcup$) e só aceitar quando encontrar o final da cadeia original.

- **Fita de Teste:** $[1][0][1][\sqcup]$

$$\delta(q_0, 0) = (q_0, \sqcup, R)$$

$$\delta(q_0, 1) = (q_0, \sqcup, R)$$

$$\delta(q_0, \sqcup) = (q_{aceita}, \sqcup, R)$$

- Rastreamento
- Diagrama de Estados

Exercício 2: Limpando a Memória

Enunciado: Crie uma MT que apague toda a entrada da fita. Ela deve substituir cada 0 ou 1 pelo símbolo branco ($\sqcup$) e só aceitar quando encontrar o final da cadeia original.

- **Fita de Teste:** $[1][0][1][\sqcup]$
- **Resultado Esperado:** A fita deve terminar completamente limpa $[\sqcup][\sqcup][\sqcup][\sqcup]$ com a máquina em q_{aceita} .

Exercício 3: O Marcador de Fim

Enunciado: Desenvolva uma MT que caminhe até o final de uma cadeia binária e, no primeiro espaço disponível (branco), escreva o símbolo #. Após escrever, a cabeça deve se mover uma posição para a esquerda e aceitar.

- **Fita de Teste:** $[0][1][1][\sqcup]$

$$\delta(q_0, 0) = (q_0, 0, R)$$

$$\delta(q_0, 1) = (q_0, 1, R)$$

$$\delta(q_0, \sqcup) = (q_{aceita}, \#, L)$$

- Rastreamento
- Diagrama de Estados

Exercício 3: O Marcador de Fim

Enunciado: Desenvolva uma MT que caminhe até o final de uma cadeia binária e, no primeiro espaço disponível (branco), escreva o símbolo #. Após escrever, a cabeça deve se mover uma posição para a esquerda e aceitar.

- **Fita de Teste:** $[0][1][1][\sqcup]$
- **Resultado Esperado:** A fita deve terminar como $[0][1][1][\#]$ com a máquina em q_{aceita} .

Exercício 4: Padronizador de Bits (Tudo "1")

Enunciado: Projete uma MT que transforme qualquer sequência em uma cadeia de apenas 1 s. Todo 0 deve ser reescrito como 1. Se o símbolo já for 1, ele deve ser mantido. A máquina aceita ao chegar no fim da cadeia.

- **Fita de Teste:** [1][0][0][1][$\sqcup$]

$$\delta(q_0, 0) = (q_0, 1, R)$$

$$\delta(q_0, 1) = (q_0, 1, R)$$

$$\delta(q_0, \sqcup) = (q_{aceita}, \sqcup, R)$$

- Rastreamento
- Diagrama de Estados

Exercício 4: Padronizador de Bits (Tudo "1")

Enunciado: Projete uma MT que transforme qualquer sequência em uma cadeia de apenas 1s. Todo 0 deve ser reescrito como 1. Se o símbolo já for 1, ele deve ser mantido. A máquina aceita ao chegar no fim da cadeia.

- **Fita de Teste:** [1][0][0][1][$\sqcup$]
- **Resultado Esperado:** A fita deve terminar como [1][1][1][1][$\sqcup$] com a máquina em q_{aceita} .

Exercício 5: O Filtro de Início

Enunciado: Crie uma MT que verifique apenas o primeiro caractere da fita.

1. Se for 0 , a máquina escreve A (de Aceito), move para a direita e entra em q_{aceita} .
2. Se for qualquer outra coisa (incluindo branco), ela entra em $q_{rejeita}$.

$$\delta(q_0, 0) = (q_{aceita}, A, R)$$

$$\delta(q_0, 1) = (q_{rejeita}, 1, R)$$

$$\delta(q_0, \sqcup) = (q_{rejeita}, \sqcup, R)$$

- **Fita de Teste A:** $[0][1][1][\sqcup]$

- **Fita de Teste B:** $[1][1][0][\sqcup]$

- Rastreamento
- Diagrama de Estados

Exercício 5: O Filtro de Início

Enunciado: Crie uma MT que verifique apenas o primeiro caractere da fita.

1. Se for $\emptyset$, a máquina escreve A (de Aceito), move para a direita e entra em q_{aceita} .
 2. Se for qualquer outra coisa (incluindo branco), ela entra em $q_{rejeita}$.
- **Fita de Teste A:** $[\emptyset][1][1][\sqcup]$ (Deve aceitar e resultar em $[A][1][1][\sqcup]$)
 - **Fita de Teste B:** $[1][1][\emptyset][\sqcup]$ (Deve rejeitar imediatamente)

Aplicação pratica da máquina de Turing

1. A Invenção do "Software" (A Máquina Universal)

Antes de Alan Turing, as máquinas eram construídas para fazer uma única coisa (como uma calculadora que só soma). Turing provou que era possível criar uma **Máquina de Turing Universal (UTM)**: uma máquina que lê a "descrição" de outra máquina na fita e a simula.

- **Onde vemos hoje:** Em tudo. Essa é a base do conceito de **computador de propósito geral**. O seu hardware não muda, mas ele pode ser um editor de texto, um jogo ou um navegador, dependendo do "manual de instruções" (software) que você carrega na memória.

2. Arquitetura de Computadores (Von Neumann)

A estrutura que usamos hoje em quase todos os dispositivos (CPU, Memória e Barramento) é uma implementação física dos conceitos de Turing.

- **Fita Infinita → RAM e SSD:** É o local onde os dados e as instruções são armazenados.
- **Cabeça de Leitura/Escrita → CPU:** O processador que busca a instrução na memória, executa a lógica e grava o resultado de volta.
- **Registro de Estados → Unidade de Controle:** Define qual instrução o processador deve executar em seguida.

3. Linguagens de Programação (Turing-Completeness)

Na ciência da computação, dizemos que uma linguagem é "**Turing-Completa**" se ela consegue simular qualquer Máquina de Turing. Isso define o poder de uma linguagem.

- **Onde vemos hoje:** Se você usa Python, JavaScript, C++ ou Java, você está usando ferramentas que têm exatamente o mesmo poder lógico que o modelo de Turing. Se algo pode ser resolvido logicamente, essas linguagens conseguem expressar isso.
- **Curiosidade:** Até ferramentas inesperadas como o **Excel**, o jogo **Minecraft** (com Redstone) e o **PowerPoint** são Turing-Completos. Isso significa que, em teoria, você poderia programar um sistema operacional inteiro dentro do Minecraft.

Resumo da Presença Atual

Conceito de Turing

Equivalente Moderno

Fita

Memória RAM / Disco Rígido

Tabela de Regras

Código Fonte / Algoritmo

Máquina Universal

O Sistema Operacional (Windows, Linux, iOS)

Mudança de Estado

Ciclo de Clock do Processador